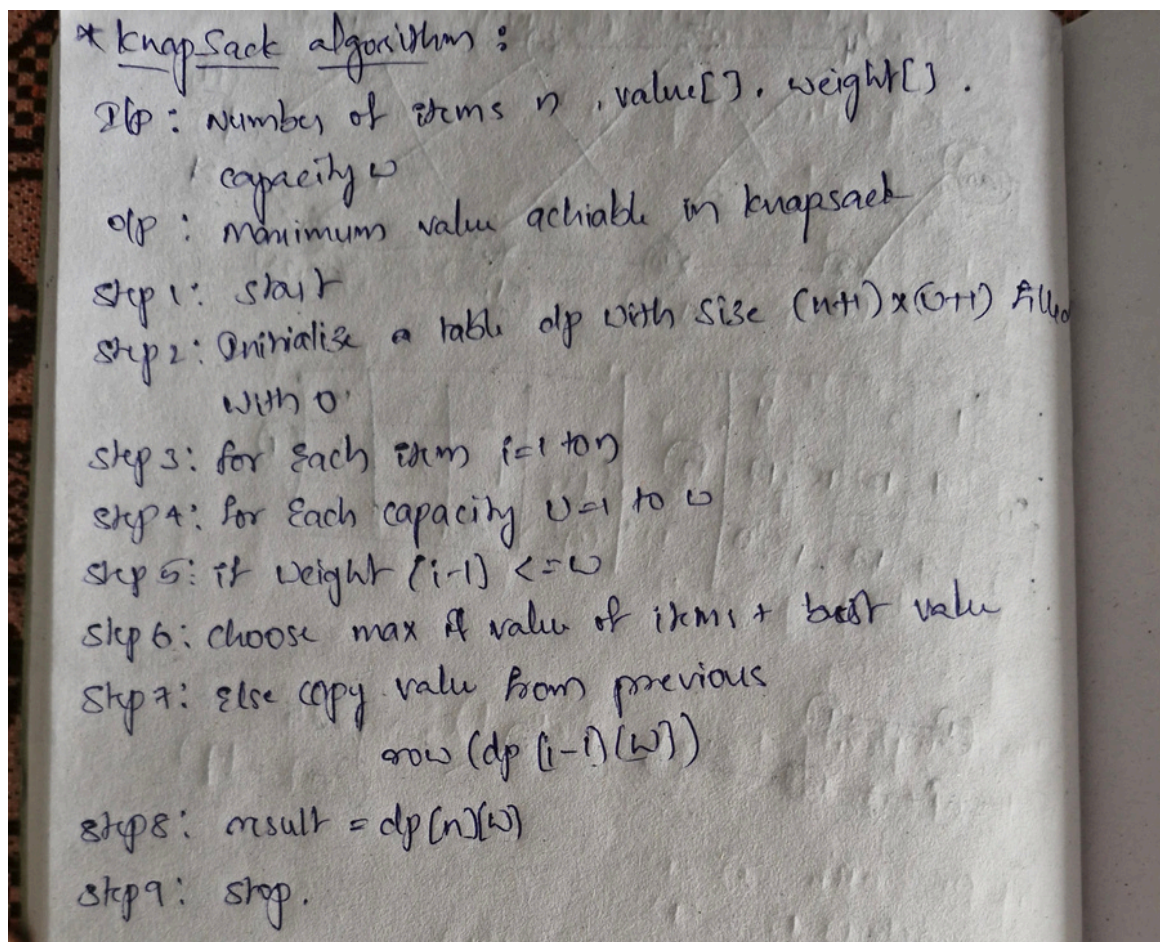


Practical: 5

Implementation of a knapsack problem using dynamic programming.

30/08/2026

Aim: Implementation of a knapsack problem using dynamic programming.**Algorithm:**

* Knapsack algorithm :

dp : number of items n , value[], weight[] .
capacity w
dp : maximum value achievable in knapsack

step 1: start
step 2: Initialize a table dp with size $(n+1) \times (w+1)$ filled with 0.
step 3: for each item $i=1$ to n
step 4: for each capacity $0=1$ to w
step 5: if weight $(i-1) \leq w$
step 6: choose max of value of items + best value
step 7: else copy value from previous row $(dp[i-1][w])$
step 8: result = $dp[n][w]$
step 9: stop.

Program:

```
#include <iostream>

#include <vector>

using namespace
std;

int knapsack(int W, vector<int>& wt, vector<int>& val, int n) {
    vector<vector<int>> dp(n+1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {
            if (wt[i - 1] <= w) {
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    return dp[n][W];
}

int main() {
    vector<int> values = {60, 100, 120}; // profits
    vector<int> weights = {10, 20, 30}; // weights
    int capacity = 50;
    int n = values.size();

    cout << "Maximum value in Knapsack = "
    << knapsack(capacity, weights, values, n) << endl;
```

```
return 0;
```

```
}
```

Output:

```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ prc5.cpp -o prc5 } ; if ($?) { .\prc5 }
Maximum value in Knapsack = 220
PS C:\Users\karth\Desktop\DSA LEARNING> |
```

Time complexity:

*Time Complexity:

for ($i=1$; $i \leq n$; $i++$)
 for ($w=1$, $w \leq W$; $w++$)
 compute $dp[i][w]$

$i=1 \rightarrow 1$
 $i \leq n \rightarrow (n+1)$ checks
 wloop $\rightarrow n * (W+1)$ checks
 $dp[i][w]$ calculation $\rightarrow n * W$ Operation

$T(n, W) = 1 + (n+1) + n(W+1) + nW$
 $= 2nW + 2n + 2$
 $= O(nW)$
 $\rightarrow$ worst case
 $\rightarrow$ Best case: $O(1)$
 $n=1$ (or) $W=1$ then
 $\rightarrow$ Average case: $O(nW)$

Conclusion:

The 0/1 Knapsack problem with dynamic programming guarantees the best value selection under weight constraints. It builds solutions step by step using subproblems to ensure optimality.